

✓ Лекция №1. Введение

П.Н. Советов, РТУ МИРЭА

```
import io
from graphviz import Digraph
from railroad import Diagram, Optional, Choice, Sequence
from IPython.display import SVG, display

def show_diagram(d):
    with io.StringIO() as f:
        Diagram(d).writeSvg(f.write)
        display(SVG(f.getvalue()))
```

О курсе

Курс программирования на языке Питон предназначен для студентов второго курса бакалавриата, уже имеющих опыт использования языков C++, Java или C#, а также знакомых с базовыми алгоритмами и структурами данных.

Акцент в курсе делается на основательном изучении языка Питон, а не на изучении популярных сегодня фреймворков. Последние за несколько лет могут устареть, поэтому полезнее иметь фундаментальную подготовку, которая позволит быстро сориентироваться и изучить актуальные библиотеки и технологии.

Задачи по курсу делятся на 3 основных типа:

1. Индивидуальные «скучные» задачи, которые генерируются (на Питоне!) процедурно для каждого студента в отдельности. Они являются обязательными к выполнению и решаются дома.
2. Задачи для практических занятий.
3. Творческие проекты, темы которых обсуждаются с преподавателями.

✓ Зачем изучать Python?

Современному программисту и так приходится иметь дело с большим количеством языков программирования. Поэтому для изучения очередного языка нужны какие-то веские причины.

Питон является одним из самых популярных и востребованных языков программирования. Об этом говорит целый ряд рейтингов за 2022-2023 гг.

- В индексе [TIOBE](#) на январь 2023 года Питон находится на первом месте.

- В рейтинге популярных языков программирования журнала [IEEE Spectrum](#) Питон находится на первом месте.
- Среди используемых языков программирования в проектах [Github](#) Питон занимает второе место.

Можно выделить следующие причины популярности Питона:

- Огромное количество библиотек для самых разных областей применения.
- Возможность быстрого прототипирования приложений благодаря наличию в языке динамической типизации, автоматического управления памятью («сборка мусора»), а также благодаря свободной поддержке различных стилей программирования.
- Легкость обучения основам языка и высокая читаемость кода. Не просто так синтаксис Питона сравнивают с «исполняемым псевдокодом».

Питон широко применяется в проектах, связанных с машинным обучением и обработкой естественного языка. На [сайте](#) популярной библиотеки для глубокого обучения TensorFlow приведены примеры использования Питона крупнейшими мировыми компаниями. Питон популярен в области анализа и визуализации данных, где используются такие известные библиотеки, как Pandas и Matplotlib.



Изображение черной дыры в центре галактики M87 получено с помощью Питона

Первое изображение черной дыры астрофизики получили с помощью инструментов, реализованных на Питоне. При этом [использовались](#) популярные библиотеки NumPy и SciPy.

Еще одной естественной областью применения Питона является разработка веб-приложений, а также извлечение данных из веб-страниц (web scrapers, web crawlers). Фреймворки Flask и Django пользуется известностью среди веб-разработчиков.

Питон пользуется популярностью в задачах системного администрирования и DevOps, а также для автоматизации тестирования. На Питоне разработана система управления конфигурациями [Ansible](#) и реализовано [автоматическое тестирование](#) для языка Haskell.

В разработке компьютерных игр Питон также используется достаточно широко. Среди известных проектов с существенной долей кода на языке Питон можно вспомнить,

например, [EVE Online](#) и [World of Tanks](#).

Как в играх, так и в более серьезных приложениях Питон часто играет роль встроенного языка для расширения функциональности программы. Питон встроен в популярные графические редакторы Gimp, Inkscape и Blender, а также в такое музыкальное ПО, как Ableton Live и Reaper.

Наконец, язык Питон сегодня является одним из самых популярных языков для обучения программированию. В этой роли он используется как в школах, так и в университетах.

✓ Что собой представляет Python

Среди русскоязычных разработчиков (и в этом тексте) одинаково употребляется и «Питон», и «Пайтон». При этом название языка никак не связано со змеями. Здесь лучше обратиться к истории создания Питона.

Автором языка является голландский программист Гвидо ван Россум. Работа над Питоном началась еще в далеком 1989 году, в стенах исследовательского института CWI в Амстердаме. Побудительным мотивом к созданию Питона было желание иметь высокоуровневый язык программирования, занимающий промежуточное положение между языком Си и высокоуровневым языком оболочки операционной системы (shell).



Автор языка Питон – Гвидо ван Россум

Название «Питон» появилось случайно. Оно происходит от названия английской комедийной передачи «Летающий цирк Монти Пайтона», популярной в том числе и среди программистов того времени. Кстати, сегодня программистов, использующих Питон, называют питонистами.

Питон унаследовал многие черты языка ABC, еще одного языка, разработанного в недрах института CWI. Сразу же обращает на себя внимание ключевая особенность синтаксиса, имеющаяся и в ABC, и в Питоне.

ABC

```
HOW TO RETURN words document:
  PUT {} IN collection
  FOR line IN document:
    FOR word IN split line:
      IF word not.in collection:
        INSERT word IN collection
  RETURN collection
```

Python

```
def words(document):
    collection = set()
    for line in document:
        for word in line.split():
            if word not in collection:
                collection.add(word)
    return collection
```

Речь, разумеется, об использовании отступов для выделения блоков в этих языках. Для программиста, который привык выделять блоки ключевыми словами `begin ... end` или фигурными скобками `{ ... }` такое решение может показаться непривычным, но в результате хорошо написанные программы на Питоне выглядят как псевдокод из учебников.

Исходные тексты первой публичной версии реализации Питона Гвидо ван Россум отправил в новостную конференцию сети Usenet. Это было в 1991 году, ниже показан перевод текста файла README. Можно заметить любопытные исторические детали, связанные с языком.

Это Питон, расширяемый интерпретируемый язык программирования, который сочетает в себе заметную мощь с очень ясным синтаксисом.

Это версия 0.9 (первая бета-версия), уровень исправления 1.

Питон можно использовать вместо сценариев оболочки, скриптов Awk или Perl, для написания прототипов реальных приложений или как язык расширения больших систем и так далее. Имеются встроенные модули, которые взаимодействуют с операционной системой и интерфейсы к различным оконным системам: оконные системы X11, Mac (для этих двух вам нужен STDWIN) и библиотека GL от Silicon Graphics. Питон работает на большинстве современных версий UNIX, на Mac и я не удивлюсь, если он будет работать в MS-DOS без изменений. я разработал язык в основном на рабочей станции SGI IRIS (с использованием IRIX 3.1 и 3.2) и на

Мас, но тестировал его также на SunOS (4.1) и BSD 4.3 (tahoe).

Собрать и установить Питон несложно (но прочтите Makefile). Страница руководства написана в стиле UNIX и имеется обширная документация (в формате LaTeX).
(В бета-версии документация все еще находится в стадии разработки.)

Пожалуйста, попробуйте поработать с ним и пришлите мне свои комментарии (на что угодно -- языковой дизайн, реализацию, переносимость, установку, документацию), а также модули, которые вы написали для Питона, чтобы сделать первый реальный релиз лучше. Если вам нужно изменить исходники, чтобы скомпилировать Питон и запустить на конкретной машине, то пришлите мне исправления -- я попробую включить их в следующий патч. Если вы не можете вообще заставить реализацию работать, то пришлите мне **подробное** описание проблемы и я попробую разобраться.

Ниже показана простая программа на языке Питон, использующая только стандартную библиотеку языка.

```
import json
from urllib.request import urlopen

def get_stars(user, repo):
    '''Получить количество звезд github-репозитория'''
    resp = urlopen(f'http://api.github.com/repos/{user}/{repo}')
    return json.loads(resp.read())['stargazers_count']

print(get_stars('true-grue', 'DandyBot'))
```

8

На примере этой компактной программы уже видно, насколько богатой является стандартная библиотека Питона (как принято говорить — *batteries included*). В данном случае используется работа по протоколу HTTP и разбор формата JSON.

При этом в программе отсутствуют объявления переменных и не указываются типы данных. Питон является динамически типизированным языком. Это означает, что проверка типов, в отличие от, к примеру, Java или C++, происходит в процессе выполнения программы, а не на стадии ее компиляции. В процессе выполнения программ на Питоне в большинстве случаев не допускается неопределенного состояния и при возникновении ошибки выполнение программы прекращается с выдачей информативного сообщения.

Питон также иногда называют динамическим языком. То есть таким языком, в котором программы во время выполнения могут изменять элементы среды выполнения языка, включая ее внутренние структуры и компилятор.

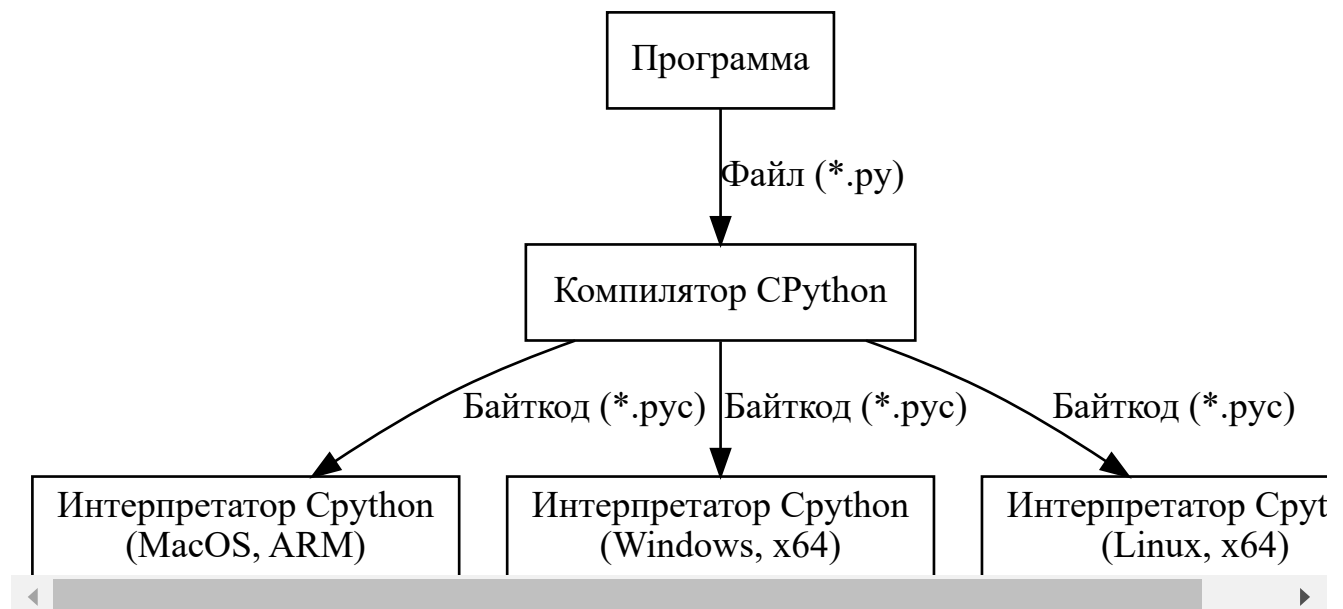
В Питоне имеется командная строка для диалога с программистом, так называемый REPL — интерактивный цикл «чтение, выполнение, вывод» (read-eval-print loop). REPL не только упрощает отладку программ, но и обеспечивает очень популярный в Питоне стиль исследовательского программирования. Этот стиль программирования удобен, когда программы развиваются эволюционно и на первых порах еще не ясны в деталях предметная область, необходимые алгоритмы и структуры данных.

В Питоне, аналогично языкам C# и Java, используется автоматическое управление памятью, то есть реализована сборка мусора.

Питон относится к мультипарадигменным языкам. В нем поддерживаются элементы процедурного, объектно-ориентированного и функционального программирования. Кроме того, имеются возможности метапрограммирования.

Иногда возникает путаница в вопросе, является ли Питон языком компилируемым или интерпретируемым. Строго говоря, язык программирования сам по себе не может быть компилируемым или интерпретируемым, это свойство реализации языка. Основная реализация Питона, CPython, написанная на Си, включает в себя и компилятор, и интерпретатор. Первым делом программа на Питоне компилируется в байткод, то есть в представление на низкоуровневом языке виртуальной машины, а затем полученное представление выполняется с помощью интерпретатора байткода.

```
d = Digraph()
d.attr('node', shape='box')
d.node('p', 'Программа')
d.node('c', 'Компилятор CPython')
d.node('i1', 'Интерпретатор Cpython\\n(MacOS, ARM)')
d.node('i2', 'Интерпретатор Cpython\\n(Windows, x64)')
d.node('i3', 'Интерпретатор Cpython\\n(Linux, x64)')
d.edge('p', 'c', label='Файл (*.py)')
d.edge('c', 'i1', label='Байткод (*.пys)')
d.edge('c', 'i2', label='Байткод (*.пys)')
d.edge('c', 'i3', label='Байткод (*.пys)')
d
```



Основные принципы, согласно которым развивается Питон, сформулированы в так называемом «Дзене Питона». Его текст на английском языке можно получить с помощью следующей команды интерпретатора CPython.

```
import this
```

Русский перевод:

Дзен Питона, Тим Питерс

- Красивое лучше, чем уродливое.
- Явное лучше, чем неявное.
- Простое лучше, чем сложное.
- Сложное лучше, чем запутанное.
- Плоское лучше, чем вложенное.
- Разреженное, лучше чем плотное.
- Читаемость имеет значение.
- Особые случаи не настолько особые, чтобы нарушать правила.
- Но практичность важнее безупречности.
- Ошибки никогда не должны замалчиваться.
- Если только они не замалчиваются явно.
- Перед лицом двусмысленности откажитесь от соблазна угадывать.
- Должен существовать один и, желательно, только один очевидный способ сделать это.
- Хотя он поначалу может быть и не очевиден, если вы не голландец. Намек на автора языка
- Лучше сейчас, чем никогда.
- Хотя никогда зачастую лучше, чем прямо сейчас.
- Если реализацию трудно объяснить, то идея плоха.

- Если реализацию легко объяснить, идея, возможно, хороша.
- Пространства имен — отличная идея! Давайте делать их больше!

Эти пункты действительно напоминают положения какого-то религиозно-философского учения и могут допускать различные толкования.

В более строгой форме развитие языка происходит в соответствии с документами [PEP](#) (Python Enhancement Proposal — предложения по развитию Питона).

✓ Недостатки языка

Питон, как и все прочие языки программирования, имеет свои границы применимости и свои слабые стороны.

Ключевым недостатком Питона можно считать практически полное отсутствие проверок корректности программы до этапа ее выполнения. Компилятор Питона своевременно оповестит программиста о синтаксических ошибках, но если вы передали в функцию, к примеру, список вместо числа, то в этом случае узнать об ошибке можно будет только во время выполнения соответствующей функции.

В Питоне, как и в других языках с динамической типизацией, указанный недостаток до некоторой степени преодолевается с помощью написания автоматических тестов и использования сторонних инструментов статического анализа. Кроме того, в современных версиях Питона введены аннотации типов, наличие которых позволяет производить необязательную статическую проверку типов (gradual typing) с использованием инструмента `mypy`.

Традиционно многие разработчики на Питоне жалуются на недостаточную производительность своих программ. Действительно, согласно [этим](#) данным, на многих задачах Питон отстает от Си более чем в 100 раз. Такой результат является платой за динамическую природу Питона. Тем не менее, существуют альтернативные реализации Питона, такие, например, как `PyPy` и `Numba`, значительно улучшающие производительность программ. Нужно, однако, добавить, что эти альтернативные реализации лишь до некоторой степени совместимы с CPython — эталонной реализацией Питона.

Еще одним серьезным недостатком Питона является недостаточная совместимость между различными версиями языка. В какой-то момент существовало две основных версии Питона — вторая и третья. Сегодня большинство разработчиков, все-таки, перешли на версию 3 (но даже в рамках этой мажорной версии существовали минорные версии языка, несовместимые друг с другом). Проблемы несовместимости все еще могут давать о себе знать в тех случаях, когда вы сталкиваетесь со старым кодом на Питоне из учебников или из сети.

Следует также добавить, что многих питонистов беспокоит регулярное добавление в язык новых конструкций, что приводит к все большему усложнению изначально простого языка.

В таблице ниже представлены основные этапы развития Питона в его третьей версии:

Версия Python 3	Главные нововведения
3.10	Сопоставление с образцом (<code>match/case</code>)
3.8	Оператор присваивания <code>:=</code> («морж»)
3.7	модуль <code>dataclasses</code> , упорядоченный <code>dict</code>
3.6	f-строки, аннотации типов для переменных, символ подчеркивания в числах, асинхронные генератор
3.5	Сопрограммы с <code>async/await</code> , оператор матричного умножения <code>@</code> , аннотации типов для функций
3.3	Конструкция <code>yield from</code> , виртуальные окружения

Установка Python

Зайдите на сайт python.org и скачайте последнюю версию Python для своей ОС. Это должна быть версия 3.10 или старше.

Если вы работаете в Linux, где уже установлен Питон, то удостоверьтесь, что обновили его до самой последней версии и сделали полную установку (по умолчанию у вас может отсутствовать стандартная GUI-библиотека Tkinter).

Войдите в командную строку ОС и удостоверьтесь, что команда `python` выдает в ответ информацию такого вида:

```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec 6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Также проверьте, что команда `pip` запускается из командной строки ОС:

```
Usage:
  pip <command> [options]
...
```

Для написания простых программ в курсе используется среда IDLE, которая входит в комплект стандартного дистрибутива Питона. Некоторые задания потребуют использования среды Jupyter Notebook.

Для работы с Jupyter-блокнотами и дополнительными библиотеками проще всего использовать редактор [Visual Studio Code](https://code.visualstudio.com/) или же можно установить отдельный дистрибутив Питона — [Anaconda](https://www.anaconda.com/).

При необходимости, можно воспользоваться [веб-реализацией](#) Питона, а также [веб-реализацией](#) Jupyter Notebook.

Если вы считаете себя уже достаточно опытным программистом, то работайте в привычных вам редакторах или IDE, однако проблемы по настройке придется решать самостоятельно. В частности, этими редакторами могут быть [Visual Studio Code](#) или [PyCharm](#).

✓ Режим калькулятора

Изучать Питон мы будем постепенно, начиная с самого простого его подмножества. Это своеобразный режим калькулятора, для работы с которым воспользуемся REPL Питона.

Традиционная программа Hello World в Питоне содержит минимум кода и не требует определения класса или функции, как в некоторых иных языках:

```
print('Привет, Python', 3)
```

```
Привет, Python 3
```

Рассмотрим задачу вычисления суммы первых n натуральных чисел по формуле:

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

```
>>> n = 5
>>> 1 + 2 + 3 + 4 + 5 # Ручной подсчет
15
>>> n * (n + 1) / 2
15.0
```

Комментарии в Питоне начинаются с символа `#`, как в примере выше, и завершаются переводом строки.

Обратите внимание, что регистр букв важен, поэтому, к примеру, переменные `n` и `N` различны. Говоря более строго, имена (или, иначе, идентификаторы) могут содержать буквы в верхнем или нижнем регистре, символ `_`, а также цифры (за исключением первого символа имени).

В Питоне имеется три основных числовых типа:

1. целые значения,
2. значения с плавающей точкой,
3. булевы значения.

Литералы (нотация, синтаксическое представление значений) целых могут быть представлены в одной из четырех систем счисления:

```
>>> 12345 # Основание 10
12345
>>> 0xDeadBeef # Основание 16
3735928559
>>> 0o757 # Основание 8
495
>>> 0b1100 # Основание 2
12
```

Числа с плавающей точкой записываются в привычной нотации с точкой, а также в экспоненциальной («научной») нотации, то есть в формате $m \times 10^n$.

```
>>> 1234.5
1234.5
>>> 12345e-1
1234.5
```

Для улучшения читаемости в числовых литералах можно использовать символ `_`:

```
>>> 1_000_000
1000000
>>> 3.14_159
3.14159
```

Целые значения могут иметь произвольный размер (точнее говоря, насколько хватит ОЗУ). Булевы значения из множества `{True, False}` являются подмножеством целых. Значения с плавающей точкой подчиняются стандарту IEEE-754 для 64-битных значений (`double` в терминологии C++).

Для преобразования чисел есть встроенные функции `int` и `float`:

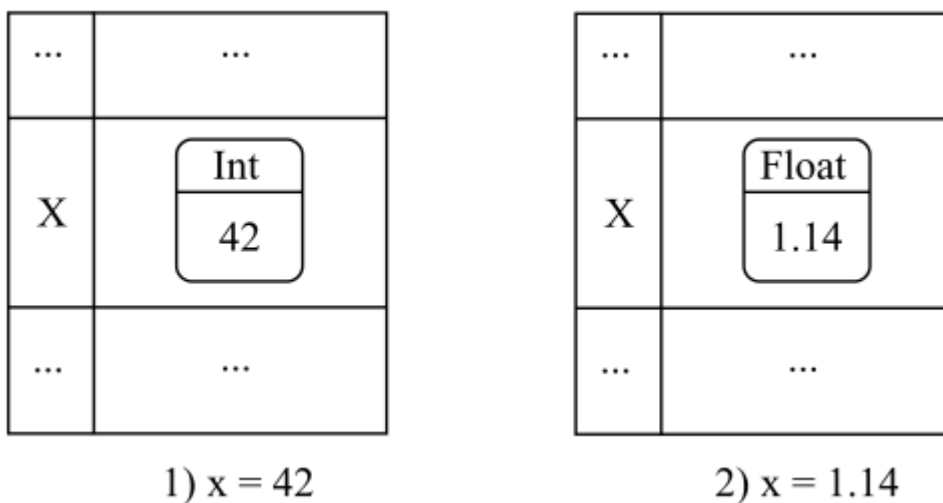
```
>>> float(4)
4.0
>>> int(2.7)
2
>>> int(True)
1
>>> int(False)
0
```

Операцию присваивания можно представить себе как запись по ключу в структуру данных словарь (ассоциативный массив, хеш-таблицу). Первое присваивание создает новую запись в этом словаре, называемом глобальным пространством имен (globals). Последующие записи по тому же ключу-имени обновляют соответствующее значение, которое может быть объектом произвольного типа.

Важно отметить, что каждое значение в Питоне является объектом-«ящиком», в котором хранится дополнительная информация, включая тип этого объекта. Узнать тип объекта можно с помощью встроенной функции `type`.

```
>>> x = 42
>>> type(x)
<class 'int'>
>>> x = 1.14
>>> type(x)
<class 'float'>
```

На рисунке ниже показано, как меняется состояние глобального пространства имен в процессе присваивания переменной `x` новых значений.



Состояние глобального пространства имен после присваиваний

С помощью встроенной функции `globals` можно узнать текущее состояние глобального пространства имен:

```
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class '_frozen_impor
```

В Питоне, как и в большинстве других языков программирования, имеются ключевые слова. Их нельзя использовать в качестве имен пользовательских объектов. Вот список ключевых слов:

```
import keyword
' '.join(keyword.kwlist)

'False None True and as assert async await break class continue def del elif else
except finally for from global if import in is lambda nonlocal not or pass raise
return try while with yield'
```

Далее показана таблица приоритетов операторов Питона, использующихся в выражениях. Строки таблицы отсортированы высшего приоритета к низшему.

Оператор	Описание
(выражение), [выражение], {ключ: значение}, {выражение}	Выражение в скобках, список, словарь, множество
x[индекс], x[индекс:индекс], x(аргументы), x.атрибут	Индексирование, срез, вызов, обращение к атрибуту
await выражение	Await-выражение
**	Возведение в степень
+x, -x, ~x	Плюс, минус, побитовое отрицание
*, @, /, //, %	Умножение, матричное умножение, деление, целочисленно
+, -	Сложение, вычитание
<<, >>	Побитовые сдвиги
&	Побитовое И
^	Побитовое исключающее ИЛИ
	Побитовое ИЛИ
in, not in, is, is not, <, <=, >, >=, !=, ==	Варианты сравнений
not x	Булево НЕ
and	Булево И
or	Булево ИЛИ
if ... else	Тернарный оператор условия
lambda	Лямбда-выражение
:=	Выражение-присваивание

Многие операции одинаково обозначаются в Питоне и C++. Операции булевой логики Питона not, and и or имеют эквиваленты в C++ в виде !, && и ||. Операция ** обозначает в Питоне возведение в степень. Операция // обозначает целочисленный вариант деления.

Операции сравнения выдают булевы значения.

В Питоне, как и в языках системного программирования, применяются побитовые операции. Зачем программисту на Питоне владеть этими операциями? Приведу несколько причин:

- Понимание исходного кода CPython.
- Реализация на Питоне виртуальных машин, симуляторов процессоров.
- Реализация специальных структур данных, использующих битовые массивы. Это может быть, например, фильтр Блума.
- Разбор двоичных форматов данных.

- Использование MicroPython для программирования микроконтроллеров в задачах из области Интернета вещей.

Дополнительные математические операции содержатся в модуле [math](#), который импортируется в текущее пространство имен, как показано ниже (о модулях мы еще будем говорить). Функции из модуля вызываются с префиксом в виде названия модуля и через точку:

```
>>> import math
>>> math.cos(0)
1.0
```

Даже в некоторых калькуляторах есть возможность создания пользовательских функций и вот простой шаблон создания пользовательских функций в Питоне:

```
def имя_функции(аргумент, ..., аргумент):
    предложение
    ...
    предложение
    return выражение
```

Не забывайте, отступы важны! В данном случае они показывают компилятору Питона, какие строки относятся к телу функции. Начинаящие также часто забывают ставить `return` перед возвращаемым результатом.

Оформим пример вычисления суммы первых натуральных чисел в виде функции:

```
def sum_first(n):
    return n * (n + 1) / 2

sum_first(5)

15.0
```

✓ Отступы и конструкции управления

Прежде чем переходить к рассмотрению ветвлений и циклов, остановимся более подробно на роли отступов в языке.

Программа на Питоне состоит из некоторого количества строк. Есть случаи, когда в Питоне объединяется несколько «физических» строк (тех, что мы наблюдаем в текстовом редакторе) в одну строку. Вот эти случаи:

- Использование символа `\` в конце строки означает ее соединение с последующей строкой.

- Выражение в круглых, квадратных или фигурных скобках занимает с точки зрения Питона одну строку, даже если элементы выражения располагаются на разных физических строках.

В Питоне, как и во многих других языках программирования, используется блочная структура. При этом блоки отмечаются исключительно позицией отступа в строке. Питонисты придерживаются следующего соглашения: отступ занимает 4 пробела.

Если вы увеличиваете отступ очередной строки, то тем самым переносите эту строку в более внутренний блок. Разумеется, строки, относящиеся к одному блоку должны обладать одинаковым отступом. Чтобы выйти из некоторого количества блоков нужно просто вернуть обратно позицию отступа до позиции желаемого блока.

Вы, вероятно уже заметили при создании собственных функций, что работать с отступами в REPL не слишком удобно. С этого момента мы переходим к записи программ в отдельных файлах.

Рассмотрим использование условного оператора `if`. Вот его шаблон:

```
if выражение:
    блок
elif выражение:
    ...
else:
    блок
```

Результат вычисления условия в `if` и `elif` трактуется, как булево значение. Ниже показан пример использования ветвлений:

```
x = -1
if x > 0:
    print('Результат положителен')
elif x < 0:
    print('Результат отрицателен')
else:
    print('Результат равен нулю')

    Результат отрицателен
```

Булевы операторы `and` и `or` в условиях выполняются слева направо до тех пор, пока не станет ясен общий результат выражения (аналогично тому, как это сделано, например, в C++). По этой причине в следующем примере ошибки не возникнет:

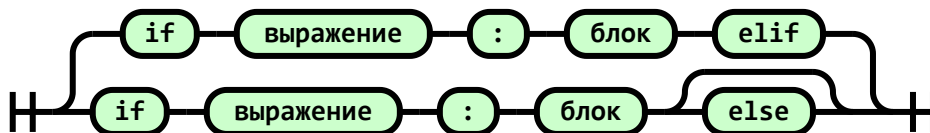
```
if 2 + 2 == 4 or 1 / 0:
    print("Деления на нуль не произошло")

    Деления на нуль не произошло
```

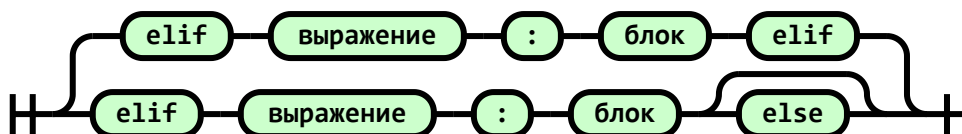
Для описания синтаксиса некоторых конструкций Питона будем использовать так называемые «железнодорожные диаграммы», читать которые нужно слева направо, мысленно двигаясь по путям и развилкам.

В более строгой форме грамматика `if` имеет следующий вид:

```
if1 = Sequence('if', 'выражение', ':', 'блок', 'elif')
if2 = Sequence('if', 'выражение', ':', 'блок', Optional('else'))
show_diagram(Choice(1, if1, if2))
```



```
elif1 = Sequence('elif', 'выражение', ':', 'блок', 'elif')
elif2 = Sequence('elif', 'выражение', ':', 'блок', Optional('else'))
show_diagram(Choice(1, elif1, elif2))
```



```
show_diagram(Sequence('else', ':', 'блок'))
```



Цикл `for` имеет следующий упрощенный шаблон:

```
for переменная in выражение:
    блок
```

В качестве выражения будем использовать вызов встроенной функции `range`, которая позволяет задать диапазон повторений блока-тела цикла:

- `range(n)`. Диапазон $[0, n)$.
- `range(n, m)`. Диапазон $[n, m)$.
- `range(n, m, s)`. Диапазон $[n, m)$ с шагом s .

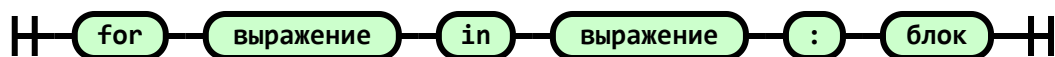
Пример использования:


```
for i in range(10):  
    print('Python', 3 + i / 10)
```

```
Python 3.0  
Python 3.1  
Python 3.2  
Python 3.3  
Python 3.4  
Python 3.5  
Python 3.6  
Python 3.7  
Python 3.8  
Python 3.9
```

Упрощенная грамматика for:

```
show_diagram(Sequence('for', 'выражение', 'in', 'выражение', ':', 'блок'))
```

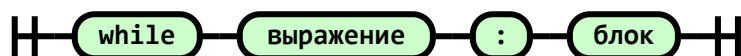


В заключение, рассмотрим шаблон для еще одного способа организации циклов — с помощью оператора `while`:

```
while выражение:  
    блок
```

Упрощенная грамматика while:

```
show_diagram(Sequence('while', 'выражение', ':', 'блок'))
```



В `for` и `while` допускается использование традиционных операторов `break` и `continue` для преждевременного выхода из цикла.

Обратите внимание, конструкции Питона, которые заканчиваются двоеточием, сигнализируют о том, что далее ожидается блок — последовательность строк, выделенных отступом. Не забывайте про двоеточие!

Литература

На русском языке

- Бизли Дэвид М. Python. Исчерпывающее руководство. — Питер, 2023. — 368 с. Современный учебник по основам языка.
- Кэвэна-Джонс Брайан, Бизли Дэвид М. Python. Книга Рецептов. — ДМК Пресс, 2019. — 646 с. Сборник разнообразных техник программирования для уже знакомых с Питоном.
- Рамальо Л. Python. К вершинам мастерства. — Litres, 2019. — 770 с. Полезные дополнительные сведения для уже знакомых с Питоном.

На английском языке

- Курс для начинающих David Beazley [Practical Python Programming](#).
- [Официальная документация](#).